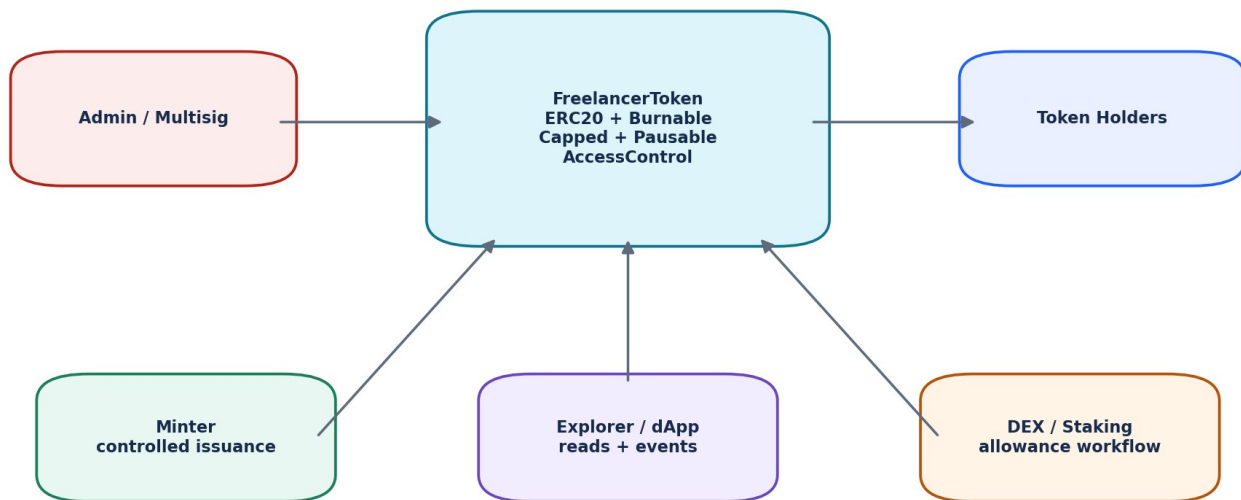


DAY 7

ERC-20 Token Development

Standards, Transfers, Allowances, Supply, Mint/Burn, Caps, Pausing, Access Control, Testing & Deployment

60-MINUTE OUTCOME Students move from writing custom Solidity state to implementing an interoperable fungible token using the ERC-20 standard and modern OpenZeppelin building blocks.



Lecture	Duration	Portfolio Outcome
Day 7	60 minutes	A role-controlled, capped, pausable, burnable ERC-20 token
Level	Beginner	Assumes Days 5-6 Solidity fundamentals
Career focus	LinkedIn + Upwork	Token creation, modification, debugging, deployment and handoff

SAFETY NOTE Educational deployment should use local networks or testnets. A production token can carry financial, security, governance, compliance and operational risk; never deploy client funds from unreviewed tutorial code.

1. Learning Objectives & Minute-by-Minute Schedule

By the end of Day 7, students should be able to explain what ERC-20 standardizes, trace direct and delegated token transfers, choose a supply model, apply access control, and test the critical behaviors of a token contract.

- Explain fungibility and why a shared token interface matters for wallets, DEXs and dApps.
- Recognize the core ERC-20 methods and Transfer / Approval events.
- Explain balanceOf, transfer, approve, allowance and transferFrom with concrete examples.
- Distinguish display decimals from raw integer token accounting.
- Explain fixed supply, mintable supply, capped supply and burning.
- Build on OpenZeppelin ERC20 instead of rewriting sensitive bookkeeping from scratch.
- Compare a single owner with granular role-based access control.
- Add minting, burning, pausing and a supply cap to a professional beginner token.
- Create a minimum test and deployment checklist before client delivery.

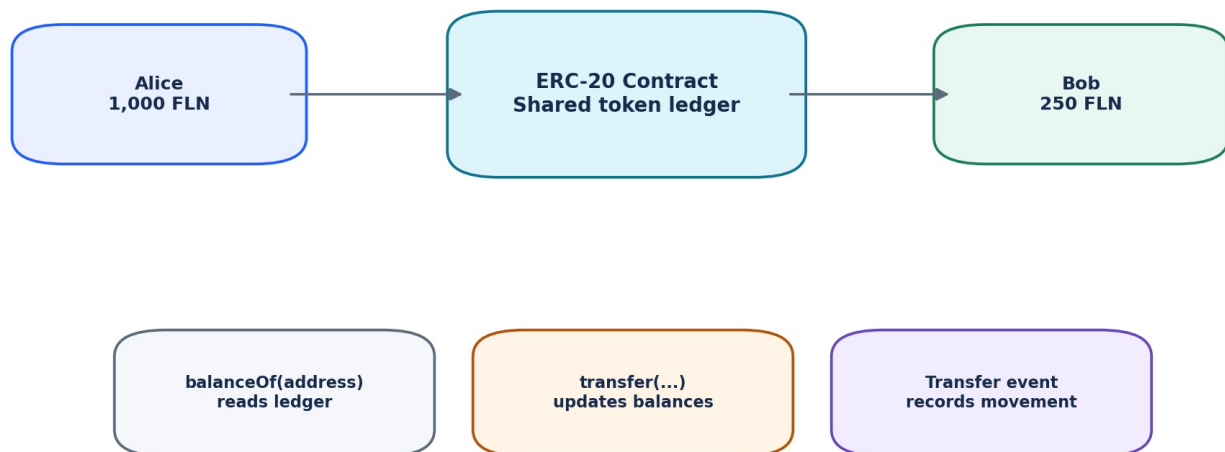
Minutes	Topic	Instructor Outcome
0-5	Token standards + fungibility	Understand the problem ERC-20 solves.
5-13	ERC-20 interface	Recognize functions, metadata and events.
13-20	Balances + direct transfers	Trace transfer state changes and logs.
20-29	Approvals + allowances	Understand approve / transferFrom delegated spending.
29-35	Decimals + supply models	Reason in raw units and choose issuance policy.
35-43	OpenZeppelin architecture	Use ERC20 and extensions correctly.
43-51	Mint/burn/cap/pause + roles	Apply security-oriented token controls.
51-57	Hands-on FreelancerToken	Read, deploy and exercise one complete contract.
57-60	Testing, quiz + homework	Check behavior and prepare portfolio work.

2. What Problem Does ERC-20 Solve? | 0-5 min

Before token standards, every token could expose a different API. Wallets, exchanges and dApps would need custom integration logic for every asset. ERC-20 standardizes a common interface for fungible tokens so applications can interact with many tokens through the same basic functions.

Fungible means interchangeable units

Asset	Fungible?	Reason
1 unit of the same ERC-20 token	Yes	One unit is intended to be equivalent to another unit of the same token.
A unique NFT #123 vs NFT #456	No	Each token can represent a distinct item/identity.
ETH	Yes	Native ETH units are interchangeable; ETH itself is not an ERC-20 contract token.



The token is accounting state inside one contract; wallets display balances by reading that state.

KEY MENTAL MODEL An ERC-20 token is usually a smart-contract ledger: the contract tracks balances and allowances. A wallet app reads this contract state; the wallet does not contain the token units inside the app.

3. ERC-20 Interface | 5-13 min

The original ERC-20 specification defines the interoperable token API. The core behavior covers supply, balances, direct transfer, delegated transfer/approval, and two standard events. name, symbol and decimals are optional metadata in the original specification but are widely used through metadata interfaces and common implementations.

Member	Purpose	Changes State?
totalSupply()	Return total token units in existence	No
balanceOf(account)	Return account token balance	No
transfer(to, value)	Move caller tokens to recipient	Yes
allowance(owner, spender)	Read remaining delegated spending amount	No
approve(spender, value)	Set spender allowance	Yes
transferFrom(from, to, value)	Spend from another account within allowance	Yes
Transfer(from, to, value)	Log token movement / mint / burn semantics	Event
Approval(owner, spender, value)	Log successful approval	Event

```
interface IERC20Minimal {
    function totalSupply() external view returns (uint256);
    function balanceOf(address account) external view returns (uint256);
    function transfer(address to, uint256 value) external returns (bool);
    function allowance(address owner, address spender) external view returns (uint256);
    function approve(address spender, uint256 value) external returns (bool);
    function transferFrom(address from, address to, uint256 value) external returns (bool);

    event Transfer(address indexed from, address indexed to, uint256 value);
    event Approval(address indexed owner, address indexed spender, uint256 value);
}
```

INTEROPERABILITY A DEX does not need to know your internal mapping names. It needs your token to behave through the standardized external interface.

4. Direct Transfers: balanceOf + transfer + Transfer | 13-20 min

A direct transfer is the simplest token movement. The caller spends their own token balance and specifies the recipient and amount.

```
// Alice calls:
token.transfer(bob, 200 * 10 ** token.decimals());

// Conceptual state change:
Alice balance: 1,000 -> 800
Bob balance:    250 -> 450
Total supply:  unchanged
```

What a correct transfer must conceptually do

1. Check that the transfer is valid according to the implementation rules.
2. Reduce the sender balance by the transferred raw token amount.
3. Increase the recipient balance by the same amount.
4. Leave totalSupply unchanged for a normal transfer.
5. Emit Transfer(from, to, value) so explorers and indexers can observe the movement.

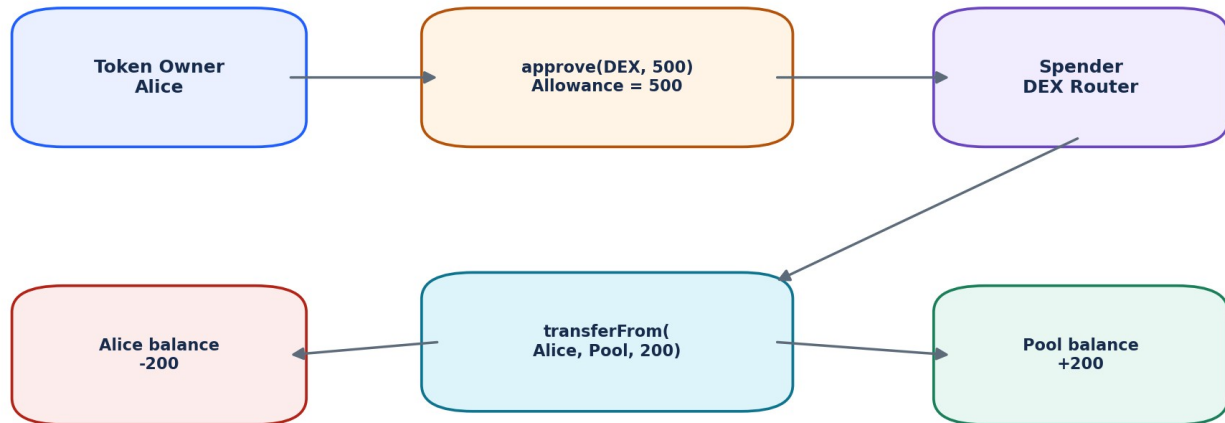
Transfer vs native ETH transfer

ERC-20 token transfer	Native ETH transfer
Calls a token contract function	Sends native value in an Ethereum transaction
Token balances change inside contract state	Native account balances change at protocol level
Requires gas paid in the chain native asset	Requires gas paid in the chain native asset
Explorer may show token Transfer logs	Explorer shows native value in transaction/value trace

CLIENT DEBUGGING If a client says "tokens are missing," first verify the token contract address, network, transaction status, Transfer event, sender/recipient, and raw amount. Do not assume the wallet UI is the source of truth.

5. Approval & Allowance: Delegated Spending | 20-29 min

Many DeFi and marketplace workflows require a smart contract to move tokens from a user. ERC-20 enables this with allowance: the token owner authorizes a spender, then that spender can call `transferFrom` within the permitted amount.



Approval grants spending authority; it does not itself transfer tokens.

Three actors to identify

Actor	Role in example
Owner	Alice owns the tokens.
Spender	DEX Router is authorized to spend some of Alice's tokens.
Recipient	Liquidity pool or another address receives the tokens.

```

// Step 1 - Alice authorizes the DEX
approve(dexRouter, 500e18);

// Step 2 - DEX spends only what is authorized
transferFrom(alice, pool, 200e18);

// Simplified result
allowance(alice, dexRouter): 500 -> 300
Alice token balance:      -200
Pool token balance:      +200
  
```

CRITICAL DISTINCTION `approve()` does not send tokens. It changes permission. `transferFrom()` is the operation that moves tokens under that permission.

6. Allowance Security & Real dApp UX

Approvals are a major real-world security surface because a user may grant a contract permission to spend tokens later. A malicious or compromised spender can use whatever allowance remains, subject to the token implementation and spender logic.

Questions to ask before approving

- Is the spender address the intended verified contract?
- How much allowance is necessary: exact amount or effectively unlimited?
- Does the application explain what the approval enables?
- Can the user reduce or revoke the allowance later?
- Is the token a standard ERC-20 or does it have unusual behavior?

Changing an existing allowance

The ERC-20 specification notes a transaction-ordering concern when changing a nonzero allowance to another nonzero value and recommends that client interfaces consider setting the allowance to zero before setting a new value. Modern systems may also use other allowance-management patterns or signed permit extensions; understand the specific token and integration rather than copy-pasting a universal workflow.

FREELANCER SCENARIO A swap UI says "insufficient allowance" although the wallet has enough tokens. Check allowance(owner, router), the actual router address, token decimals/raw units, and whether a previous approval targeted a different contract.

Permit - preview of an advanced extension

ERC-2612 adds permit(), which lets a token owner authorize an allowance using a signed message rather than sending the normal approve transaction. We introduce the concept today; signature-domain details belong in a later advanced lesson.

7. Decimals & Raw Units | 29-32 min

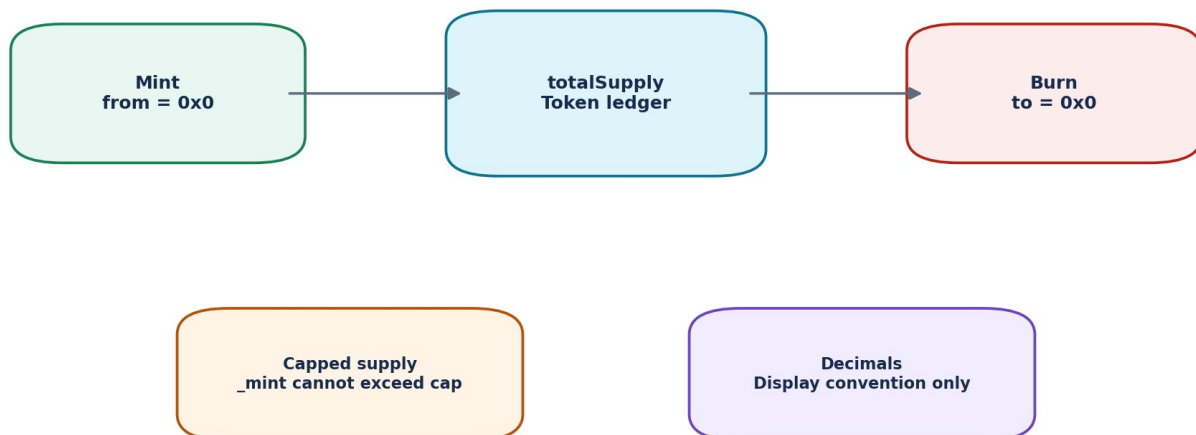
Solidity performs integer arithmetic. Token decimals are primarily a display convention that tells user interfaces how to present raw integer balances.

```
// If decimals() == 18:
1 token      = 1_000_000_000_000_000_000 raw units
10 tokens    = 10 * 10 ** 18 raw units
0.5 token    = 500_000_000_000_000_000 raw units
```

Raw balance	decimals	UI display
505	2	5.05
1,000,000	6	1.000000
1,000,000,000,000,000,000	18	1.0

COMMON BUG "Mint 1,000 tokens" is ambiguous in Solidity. If the token uses 18 decimals, minting the integer 1000 creates only 1000 raw units, displayed as 0.0000000000000001 tokens. Convert human amounts deliberately.

8. totalSupply & Supply Models | 32-35 min



Supply design is a business/security decision layered on top of the ERC-20 interface.

Supply Model	How It Works	Typical Requirement
Fixed supply	Mint once, no future public/admin mint path	Simple capped asset with predetermined supply
Mintable	Authorized role can create new units	Rewards, emissions, bridges, operations
Capped mintable	Minting allowed but total supply cannot exceed cap	Controlled issuance with hard upper bound
Burnable	Holders (and possibly approved spenders) destroy units	Redemption or supply reduction mechanics

9. Minting and Burning Are Not Magic ERC-20 Features

The base ERC-20 interface does not specify a public mint function. A derived implementation decides how supply is created. In OpenZeppelin ERC20, the internal `_mint` function is used by your contract's chosen supply mechanism. Burning similarly reduces a balance and total supply through controlled logic.

Mint semantics

```
function mint(address to, uint256 amount)
    external
    onlyRole(MINTER_ROLE)
{
    _mint(to, amount);
}
```

- Recipient balance increases.
- `totalSupply` increases.
- A Transfer event is represented from the zero address to the recipient.
- The public/external permission controlling mint is your design responsibility.

Burn semantics

```
// With ERC20Burnable:
burn(amount); // holder burns own tokens
burnFrom(account, amount); // caller burns account tokens within allowance
```

- The burned account balance decreases.
- `totalSupply` decreases.
- A Transfer-style event represents movement to the zero address.
- Burning does not send tokens to a normal wallet; it removes units from supply accounting.

CLIENT REQUIREMENT QUESTION When a job says "create a token," ask: fixed or mintable supply? Who can mint? Is there a cap? Can holders burn? Is pausing required? Who controls admin roles? These are architecture decisions, not cosmetic settings.

10. Why Use OpenZeppelin Contracts? | 35-43 min

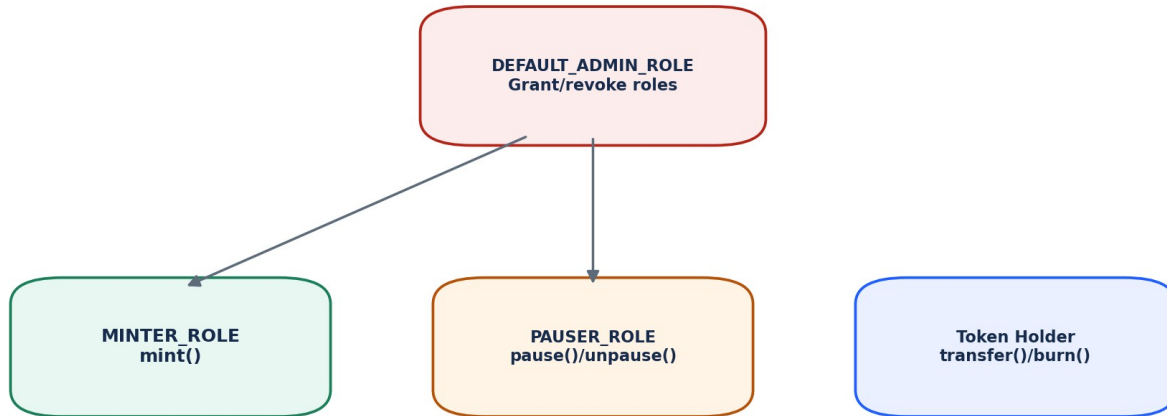
For professional work, avoid hand-writing the core ERC-20 bookkeeping unless the project specifically requires a custom implementation and you can justify/audit it. OpenZeppelin Contracts provides widely used modular implementations and extensions.

Building Block	Purpose
ERC20	Core ERC-20 implementation; supply mechanism is added by derived contract.
ERC20Burnable	Allows holders to burn their own tokens and allowance-based burning.
ERC20Capped	Enforces an immutable maximum total supply during minting.
ERC20Pausable	Adds a pause check affecting transfers, minting and burning; you must expose controlled pause/unpause functions.
AccessControl	Granular role-based permissions such as MINTER_ROLE and PAUSER_ROLE.
Ownable / Ownable2Step	Simpler single-owner administration when granular roles are unnecessary.
ERC20Permit	Signed approvals through ERC-2612; advanced topic.
SafeERC20	Utility wrappers for safely interacting with token contracts from another contract.

VERSION AWARENESS OpenZeppelin Contracts 5.x centralizes transfer/mint/burn customization around the internal `_update` function. Code written for older OpenZeppelin versions may override different hooks and may not compile unchanged. Always identify the dependency version before editing a client contract.

11. Ownership vs Role-Based Access Control | 43-46 min

A single owner is easy to reason about. Role-based access becomes useful when different operational accounts should have different powers.



Separate roles reduce the power held by any single operational account.

Pattern	Strength	Trade-off
Ownable	Simple, small surface area, easy admin model	One owner may have too much authority; ownership transfer must be handled carefully.
AccessControl	Separate minter, pauser, admin and other permissions	More configuration and role-admin relationships to test/operate.
Multisig as admin	Reduces dependence on one private key	Operational coordination and multisig configuration required.

LEAST PRIVILEGE A pauser does not need mint permission. A minter does not necessarily need the power to grant roles. Separate permissions according to responsibility.

12. Pausing & Capping | 46-51 min

Supply cap

A cap limits the maximum totalSupply that minting can create. This is useful only if the client actually wants a hard maximum; do not add a cap merely because it sounds safer.

Emergency pause

Pausable token logic can stop token movements during an incident or planned control period. But pausing is itself privileged power. The client should understand who can pause, how unpausing works, and the centralization implications.

```
function pause() external onlyRole(PAUSER_ROLE) {
    _pause();
}

function unpause() external onlyRole(PAUSER_ROLE) {
    _unpause();
}
```

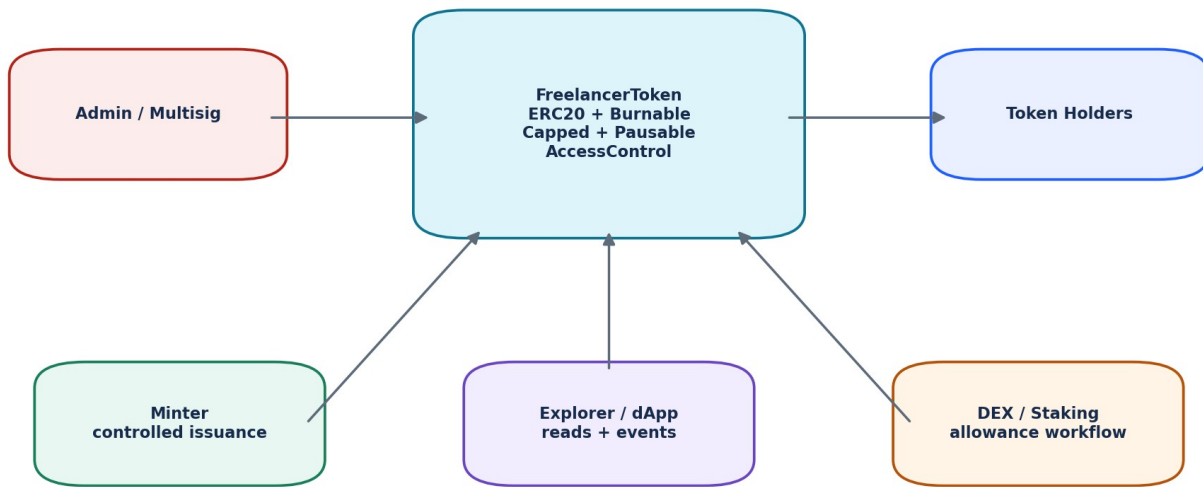
Admin handoff questions

- Which address or multisig receives DEFAULT_ADMIN_ROLE?
- Who receives MINTER_ROLE and what operational process authorizes minting?
- Who receives PAUSER_ROLE and what incident criteria justify using it?
- Can roles be revoked or reassigned, and who has authority to do so?
- Does the README clearly disclose the cap, mint ability, burn behavior and pause authority?

DESIGN DISCUSSION "Can the owner freeze all transfers?" is not only a coding question. It affects user expectations, decentralization, operational security and sometimes legal/compliance requirements. Document privileged capabilities in the client handoff.

13. Hands-On Project: FreelancerToken | 51-57 min

This educational contract combines the concepts in one project. It uses OpenZeppelin Contracts 5.x-style inheritance and exposes controlled minting and pausing. Token holders can burn their own tokens. The cap limits future minting.



```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC20Burnable} from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import {ERC20Capped} from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Capped.sol";
import {ERC20Pausable} from "@openzeppelin/contracts/token/ERC20/extensions/ERC20Pausable.sol";
import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";

contract FreelancerToken is
    ERC20,
    ERC20Burnable,
    ERC20Capped,
    ERC20Pausable,
    AccessControl
{
    bytes32 public constant MINTER_ROLE = keccak256("MINTER_ROLE");
    bytes32 public constant PAUSER_ROLE = keccak256("PAUSER_ROLE");

    constructor(
        address admin,
        uint256 initialSupply,
        uint256 supplyCap
    )
        ERC20("Freelancer Token", "FLN")
        ERC20Capped(supplyCap)
    {
        _grantRole(DEFAULT_ADMIN_ROLE, admin);
        _grantRole(MINTER_ROLE, admin);
        _grantRole(PAUSER_ROLE, admin);
        _mint(admin, initialSupply);
    }
}
```

14. FreelancerToken - Functions & Multiple-Inheritance Override

```

function mint(address to, uint256 amount)
    external
    onlyRole(MINTER_ROLE)
{
    _mint(to, amount);
}

function pause() external onlyRole(PAUSER_ROLE) {
    _pause();
}

function unpause() external onlyRole(PAUSER_ROLE) {
    _unpause();
}

function _update(address from, address to, uint256 value)
    internal
    override(ERC20, ERC20Capped, ERC20Pausable)
{
    super._update(from, to, value);
}
}

```

Why is _update overridden?

Both ERC20Capped and ERC20Pausable customize the ERC20 update path. Solidity requires the derived contract to resolve the multiple-inheritance override and call super so each extension in the inheritance linearization can enforce its behavior. In OpenZeppelin 5.x, _update is the key customization point for transfer, mint and burn behavior.

Deployment input example

```

// Human design goal:
Initial supply: 1,000,000 FLN
Cap:           10,000,000 FLN
Decimals:      18 (OpenZeppelin default)

// Raw constructor values:
initialSupply = 1_000_000 * 10**18
supplyCap     = 10_000_000 * 10**18

```

DEPLOYMENT CHECK admin must not be an accidental address. For real projects, the admin may be a multisig rather than the deployer EOA. Verify the address, network, dependency version and constructor units before broadcasting.

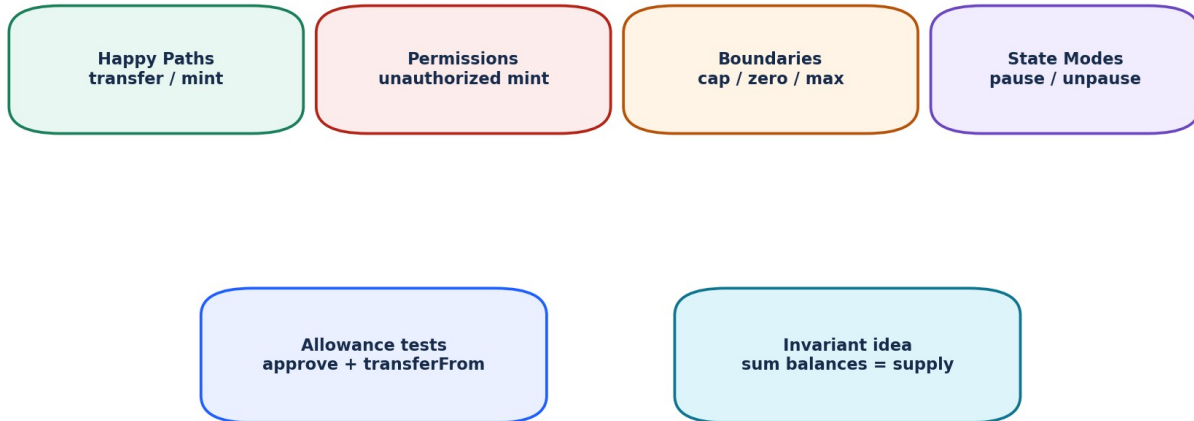
15. Hands-On Lab: Remix / Foundry / Hardhat Checklist

1. Create a clean project using the environment prepared on Day 4.
2. Install or import the intended OpenZeppelin Contracts version. Do not mix code examples from different major versions.
3. Compile FreelancerToken and read every compiler error instead of suppressing it.
4. Deploy first to a local network or test environment with a known admin test account.
5. Call name(), symbol(), decimals(), totalSupply(), cap() and balanceOf(admin).
6. Transfer tokens from the admin to a second test account.
7. Approve a spender and exercise transferFrom with a smaller amount.
8. Mint within the cap using an account that has MINTER_ROLE.
9. Attempt mint from an unauthorized account and confirm it reverts.
10. Pause the token and confirm a transfer reverts; unpause and confirm transfer succeeds.
11. Burn a portion of a holder balance and confirm totalSupply decreases.
12. Attempt minting above the cap and confirm the transaction reverts.

Record the evidence

Check	Expected Evidence
Deployment	Contract address + successful receipt
Transfer	Balances changed + Transfer log
Approval	allowance(owner, spender) returns expected value
transferFrom	Allowance/balances change correctly
Unauthorized mint	Revert / failed test
Pause	Transfer fails while paused
Cap	Mint that exceeds cap fails
Burn	Holder balance and totalSupply decrease

16. Testing Strategy | 57-58 min



A token is not finished when it compiles; it is finished when behavior and permissions are tested.

Minimum test cases for a client token

Area	Test
Metadata	name, symbol, decimals are expected.
Initial supply	Initial supply assigned to intended address.
Transfer	Balances and Transfer log are correct.
Insufficient balance	Transfer above balance reverts.
Allowance	approve stores allowance; transferFrom spends within it.
Allowance limit	transferFrom above allowance reverts.
Mint permissions	Only MINTER_ROLE can mint.
Cap	Mint cannot make totalSupply exceed cap.
Pause	Transfers/mint/burn behavior matches pause design.
Burn	Burn reduces balance and supply.
Roles	Grant/revoke/admin behaviors are tested intentionally.

TESTING MINDSET Do not only prove that authorized actions work. Prove that unauthorized and boundary actions fail exactly where expected.

17. Common ERC-20 Mistakes & Debugging

Symptom	Likely Cause	First Check
Wallet shows tiny balance	Raw amount not scaled by decimals	decimals() and raw balance
transferFrom reverts	Insufficient/wrong allowance	allowance(owner, msg.sender/spender)
Mint fails unexpectedly	Supply cap reached or paused	totalSupply, cap, paused state
Mint works for wrong account	Bad role configuration	hasRole + role grants/events
Pause button works but transfer still works	Wrong implementation/version/override	dependency version and _update override
Token does not appear in wallet	UI discovery issue or wrong network/address	network + token contract + balanceOf
DEX cannot pull token	Approval targets wrong router or nonstandard behavior	spender address + allowance + token behavior
Contract does not compile	Imports/version mismatch or override list	package version + compiler message

Debug workflow

1. Confirm chain ID and token contract address.
2. Identify the exact failing function and caller.
3. Read balances, allowances, roles, paused state, total supply and cap.
4. Inspect transaction input, receipt and logs.
5. Reproduce the behavior locally/testnet with the same dependency version.
6. Write a regression test before applying the fix.
7. Document whether the issue is contract logic, permissions, integration or UI display.

18. Upwork / LinkedIn Client Scenarios

Scenario A - "Create a 100M token"

Do not immediately code. Clarify name, symbol, decimals, fixed/mintable supply, initial recipient, cap, burn, pause, roles, admin/multisig, network, deployment/verification, testing and ownership handoff.

Scenario B - "Add mint function to our token"

First identify the existing implementation, proxy/upgradeability status, OpenZeppelin version, storage/upgrade constraints, current access model and who should receive mint authority. Adding a public mint function without access control would be catastrophic.

Scenario C - "Users cannot swap our token"

Check whether the issue is allowance, wrong router, pause state, transfer restrictions, fee-on-transfer/custom token behavior, liquidity availability, decimals/display or frontend integration. Do not assume the DEX is broken.

Scenario D - "Renounce ownership so it is safe"

Explain consequences first. Renouncing ownership can permanently disable owner-only operations. If the token uses AccessControl, ownership may not even be the relevant authority. Map all privileged roles before making irreversible changes.

PROPOSAL LANGUAGE A strong freelancer proposal says what you will verify: token requirements, supply/permission model, standard implementation, tests, testnet deployment, explorer verification and admin handoff. Avoid promising "100% secure" or "bug-free."

19. Interview Questions + Quick Answers

Question	Strong Beginner Answer
What is ERC-20?	A standard interface/behavior for fungible token contracts so wallets and applications can integrate consistently.
transfer vs transferFrom?	transfer spends caller tokens; transferFrom spends from another owner using the caller's allowance.
Does approve transfer tokens?	No. It changes a spender allowance.
What is allowance?	The remaining amount a spender may transfer from an owner through transferFrom.
What do decimals do?	They define user-facing display scaling; contract accounting still uses integers.
Does ERC-20 require mint()?	No. The implementation chooses a supply mechanism.
What is ERC20Capped?	An extension that enforces a maximum total supply on minting.
Why role-based access?	To give separate accounts only the permissions they need, such as mint vs pause.
What happens when tokens burn?	Balance and total supply decrease; the movement is represented through zero-address semantics.
Why use OpenZeppelin?	To reuse established modular standard implementations instead of reimplementing sensitive token accounting unnecessarily.

Two deeper interview questions

- Why can an unlimited allowance be dangerous? Because a spender retains broad future authority to move the owner's tokens until the allowance is reduced/revoked or otherwise constrained.
- Why can old token code fail after upgrading OpenZeppelin? Major versions may change internal extension hooks and constructors; dependency versions must match the code architecture.

20. Day 7 Mini Quiz | 58-60 min

1. What problem does ERC-20 standardization solve?
2. Which function reads an account token balance?
3. Does transfer change totalSupply during a normal transfer?
4. What is the difference between approve and transferFrom?
5. If decimals is 18, how many raw units represent 1 token?
6. Is mint() required by the ERC-20 standard?
7. What does a capped token enforce?
8. Why would a project separate MINTER_ROLE from PAUSER_ROLE?
9. What should happen to totalSupply when tokens are burned?
10. Name three checks you should perform before deploying a client token.

Answer Key

1. A common API/behavior so wallets, exchanges and dApps can integrate fungible tokens consistently.
2. balanceOf(address).
3. No; balances move but total supply stays the same.
4. approve sets delegated spending permission; transferFrom uses that permission to move tokens.
5. 10^{18} raw units.
6. No; the token implementation defines its supply mechanism.
7. A maximum total supply during minting.
8. Least privilege: each operational account gets only the power it needs.
9. It decreases by the burned amount.
10. Examples: network/chain, admin address, raw constructor units, supply/cap, dependency/compiler version, role design, tests.

21. Homework & Portfolio Assignment

Task 1 - Build three variants

- FixedSupplyToken: mint the entire supply in the constructor and expose no future mint function.
- MintableRoleToken: add MINTER_ROLE and test unauthorized mint failure.
- ControlledToken: use mint + burn + cap + pause + separate role permissions.

Task 2 - Write at least 10 tests

Cover successful transfers, insufficient balance, approval, transferFrom, allowance failure, authorized/unauthorized mint, cap failure, pause behavior and burn supply reduction.

Task 3 - Transaction analysis

On a test network, perform one transfer and one approval. Record transaction hash, caller, token contract, function, raw amount, status, gas fee, emitted event and final balances/allowance.

Task 4 - Client requirements exercise

Write a one-page specification for a fictional client requesting "a 10 million token". Explicitly define name, symbol, decimals, supply model, cap, mint/burn/pause permissions, admin custody, network, deployment, verification, tests and handoff.

PORTFOLIO STANDARD Your GitHub README should explain behavior and privileged roles, include test commands, deployment parameters, sample testnet address, verification steps and a clear warning that test deployments are not audited production systems.

22. Printable Day 7 Cheat Sheet

Concept	Remember
ERC-20	Common fungible-token interface/behavior.
balanceOf	Read token units owned by an address.
transfer	Caller sends own tokens.
approve	Owner grants spender an allowance.
allowance	Remaining delegated amount.
transferFrom	Spender moves owner tokens within allowance.
Transfer event	Logs token movement; zero-address semantics represent mint/burn.
Approval event	Logs successful approval.
decimals	Display scaling convention, not fractional Solidity arithmetic.
totalSupply	Raw units currently in existence.
mint	Implementation-defined creation of new units.
burn	Destroys units; balance and total supply decrease.
cap	Maximum total supply enforced during minting.
pause	Emergency/operational control; privileged and centralizing.
AccessControl	Granular roles such as MINTER_ROLE and PAUSER_ROLE.
OpenZeppelin	Modular implementations/extensions; match code to package version.
OZ 5.x _update	Primary customization path for transfers/mints/burns in current 5.x ERC20.

The two ERC-20 flows to memorize

DIRECT TRANSFER

Owner -> token.transfer(recipient, amount) -> balances update -> Transfer event

DELEGATED TRANSFER

Owner -> approve(spender, amount) -> allowance

Spender -> transferFrom(owner, recipient, amount) -> balances + allowance update

DAY 7 COMPLETION STANDARD Before Day 8, you should be able to deploy a test ERC-20, trace transfers and allowances, explain raw units, add a controlled supply mechanism, and prove permissions/boundaries with tests.

23. Instructor Whiteboard Flow & Day 8 Preview

Whiteboard flow for the 60-minute lecture

```

Token standard problem
  |
  v
ERC-20 interface
  |
  +--> transfer -----> balances + Transfer event
  |
  +--> approve --> allowance --> transferFrom
  |
  v
Decimals + totalSupply
  |
  v
Supply design: fixed / mint / burn / cap
  |
  v
Access design: owner / roles / multisig
  |
  v
OpenZeppelin extensions
  |
  v
Tests -> testnet deploy -> verify -> client handoff

```

Day 8 Preview - NFT Development

Day 8 moves from fungible units to non-fungible and multi-token standards: ERC-721, ERC-1155, token IDs, ownership, approvals, metadata, tokenURI, IPFS concepts, minting and a portfolio NFT collection.

Primary Technical References

- Ethereum Improvement Proposals - ERC-20: Token Standard (EIP-20).
- OpenZeppelin Contracts 5.x - ERC20 API and extensions.
- OpenZeppelin Contracts 5.x - Access Control guide.
- Ethereum Improvement Proposals - ERC-2612 Permit Extension for signed approvals.
- Solidity documentation - contracts, inheritance, events and error handling.

REFERENCE NOTE The lecture intentionally teaches the stable ERC-20 mental model and marks version-sensitive OpenZeppelin 5.x details. For client work, always check the exact dependency commit/version in the repository before modifying hooks or inheritance.